LAPORAN PRAKTIKUM
INTERNET OF THINKS 2



Oleh:

Nama : Salsabila Aulia Zahra

Npm : 25782111

Kelas : 3D

Dosen:

Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T.

Agus Ambarwari, S.Pd., M.Kom.

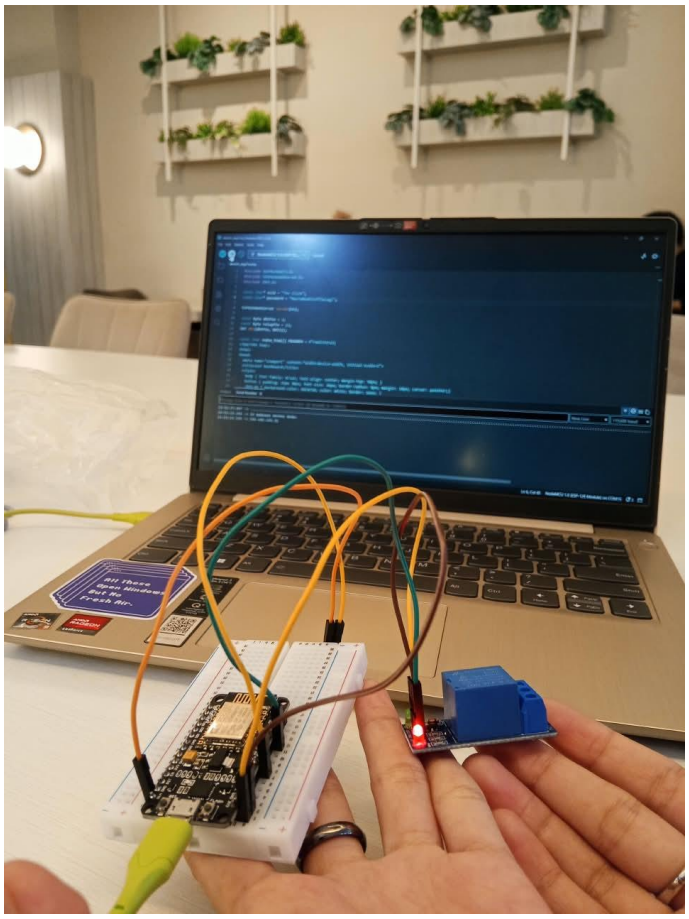
TEKNOLOGI REKAYASA INTERNET

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI LAMPUNG

2026/2027

Praktikum 1: NodeMCU sebagai *Web Server* (Menyajikan HTML)



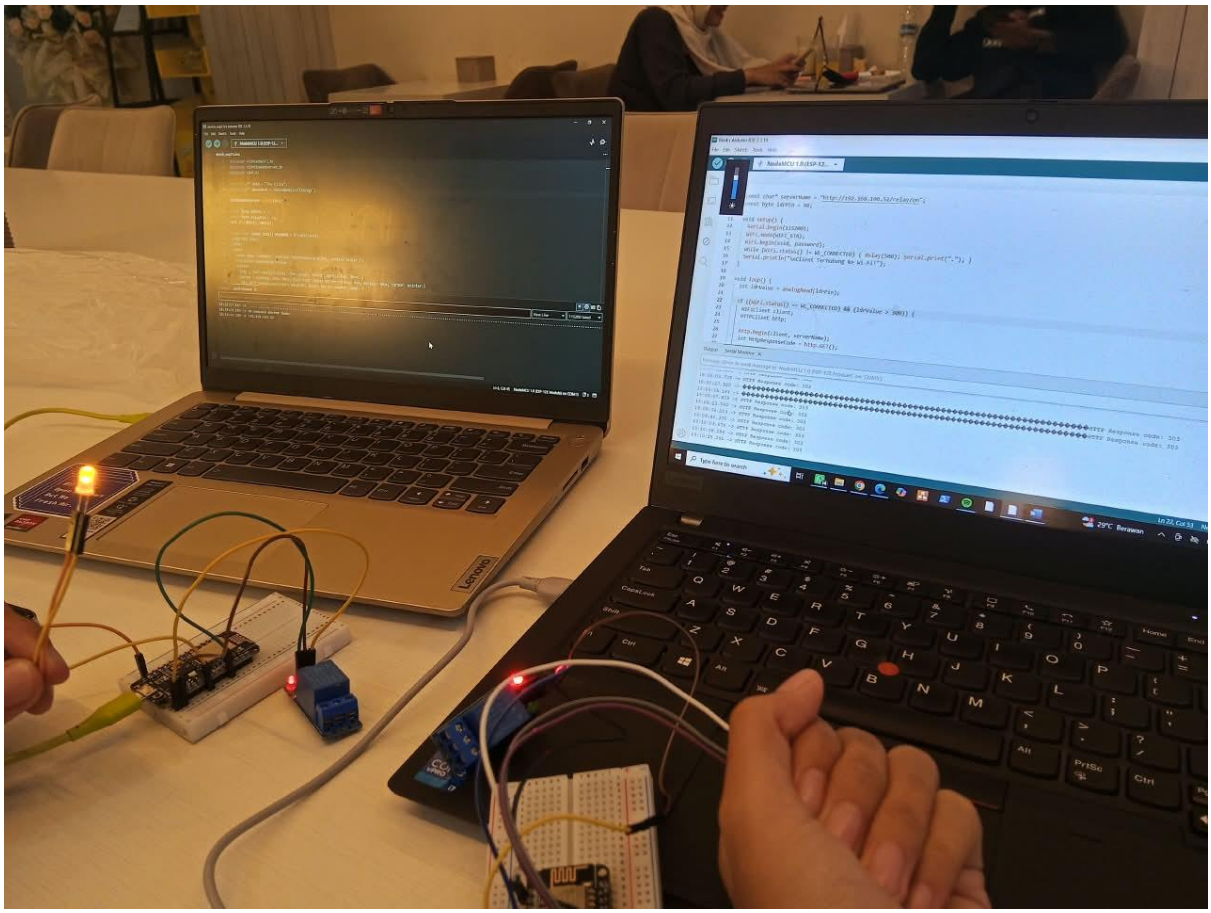
Langkah Kerja Rangkaian:

1. Sensor DHT dipasang pada NodeMCU dengan menghubungkan pin DATA/OUT/S ke **D4 (GPIO 2)**.
2. Pin VCC sensor DHT disambungkan ke **3V3**, sedangkan pin GND disambungkan ke **GND** NodeMCU.
3. Untuk bagian output, LED dipasang pada pin **D6 (GPIO 12)**.
4. Kaki LED yang terhubung ke D6 digunakan sebagai jalur keluaran, sedangkan kaki lainnya dihubungkan ke resistor **220 Ohm** dan kemudian ke GND.
5. Setelah semua komponen terpasang, dilakukan pengecekan kembali agar tidak ada kabel yang salah posisi.
6. Pada percobaan ini relay diganti menggunakan LED agar proses pengujian dengan NodeMCU Client dapat dilakukan dengan lebih mudah.

Hasil Praktikum 1

Setelah rangkaian selesai dibuat dan program dijalankan, NodeMCU dapat terhubung ke jaringan Wi-Fi dan bekerja sebagai Web Server. Alamat IP yang diperoleh dari Serial Monitor digunakan untuk membuka halaman Web Server melalui browser. Pada halaman tersebut dapat dilihat informasi suhu dan tersedia tombol untuk mengatur kondisi LED.

Praktikum 2: NodeMCU sebagai HTTP Client (Meminta Layanan)



Langkah Kerja Rangkaian

1. Rangkaian dari percobaan sebelumnya digunakan kembali sebagai dasar.
2. Sensor DHT dilepas dari NodeMCU Client karena pada percobaan ini digunakan sensor LDR.
3. Sensor LDR dirangkai bersama resistor **10K Ohm** sehingga membentuk pembagi tegangan.
4. Bagian keluaran dari rangkaian LDR dihubungkan menuju pin **A0** NodeMCU.
5. Setelah rangkaian selesai, seluruh kabel diperiksa kembali untuk memastikan pemasangannya sudah benar.
6. NodeMCU Client kemudian disiapkan agar dapat berkomunikasi dengan NodeMCU Server melalui jaringan Wi-Fi yang sama.

Hasil Praktikum 2

Pada percobaan ini NodeMCU Client berhasil membaca perubahan kondisi cahaya menggunakan LDR. Ketika kondisi cahaya memenuhi batas yang ditentukan pada program, Client mengirimkan HTTP GET Request menuju NodeMCU Server. Server kemudian menerima permintaan tersebut dan mengubah kondisi LED.

- **Latihan 1 (Pemahaman Jaringan):** Berdasarkan wawasan dari subbab 3.3.1, sebutkan secara logis mengapa kita menggunakan mode jaringan *Station (STA)* pada kedua praktikum di atas, dan bukan mode *Access Point (AP)*?

  Jawab:

  Mode Station (STA) digunakan karena NodeMCU perlu masuk ke jaringan Wi-Fi yang sudah tersedia. Dengan cara ini, NodeMCU Server dan NodeMCU Client dapat berada dalam jaringan yang sama sehingga keduanya bisa saling berkomunikasi.

  Jika menggunakan mode Access Point (AP), NodeMCU justru membuat jaringan Wi-Fi sendiri. Hal tersebut tidak sesuai dengan rancangan percobaan karena komunikasi pada praktikum menggunakan jaringan Wi-Fi yang sudah ada.

  Jadi, penggunaan STA dipilih agar NodeMCU dapat terhubung ke jaringan yang sama dan melakukan komunikasi antara Client dan Server.

- **Latihan 2 (Modifikasi Antarmuka & Telemetri):** Pada latihan ini dilakukan perubahan pada tampilan Web Server. Data yang sebelumnya hanya menampilkan suhu ditambahkan dengan informasi kelembapan. Selain itu, tombol ON dan OFF digabung menjadi satu tombol yang dapat menyesuaikan keadaan LED.

  Penambahan Kelembapan

  Pembacaan kelembapan dapat dilakukan dengan:

  float humidity = dht.readHumidity();

  Pada bagian HTML ditambahkan;

  Kelembapan: %HUMIDITY% %

  Kemudian nilai kelembapan dimasukkan ke halaman menggunakan:

  html.replace("%HUMIDITY%", String(humidity));

  Pembuatan Tombol Toggle

  Pada HTML dapat dibuat placeholder:

  %RELAY_BUTTON%

  Kemudian status LED diperiksa menggunakan:

  digitalRead(relayPin)

  Jika LED sedang mati, tombol diarahkan ke:

  /relay/on

  Jika LED sedang menyala, tombol diarahkan ke:

  /relay/off

  Jawab

Dengan perubahan tersebut, tampilan Web Server menjadi lebih informatif karena tidak hanya menunjukkan suhu tetapi juga kelembapan. Penggunaan satu tombol juga membuat pengendalian LED menjadi lebih praktis karena tulisan dan fungsi tombol dapat mengikuti kondisi LED saat itu.

Modifikasi ini tetap menggunakan konsep placeholder pada HTML dan pengecekan kondisi pin LED seperti yang diminta pada modul.

```
  <div class="sensor-box">

    <p>
      Suhu Saat Ini:
      <strong>%TEMPERATURE%</strong>
      Celcius
    </p>

    <p>
      Kelembapan:
      <strong>%HUMIDITY%</strong>
      %
    </p>

  </div>

  <h2>Kendali Relay</h2>

  %RELAY_BUTTON%

</body>

</html>
)rawliteral";
```

```
if (isnan(t)) {

  html.replace("%TEMPERATURE%", "--");

} else {

  html.replace("%TEMPERATURE%", String(t));

}


// ========================
// Menampilkan kelembapan
// ========================

if (isnan(h)) {

  html.replace("%HUMIDITY%", "--");

} else {

  html.replace("%HUMIDITY%", String(h));

}
```

Latihan 3 (Analisis Kode Client): Apa yang terjadi jika NodeMCU Server tiba-tiba mati atau kehilangan daya?

Jawab

Jika NodeMCU Server dalam keadaan mati, Client tidak akan mendapatkan balasan dari Server ketika menjalankan perintah http.GET().

Pada Serial Monitor akan terlihat kode error, misalnya:

HTTP Response code: -1

Hal tersebut menunjukkan bahwa permintaan dari Client tidak mendapatkan respons dari Server karena Server sedang tidak dapat diakses.

Latihan 4 (Pengembangan Algoritma): Bagaimana mengubah kondisi agar Client hanya mengirim GET Request ketika suhu lebih dari 35°C?

Jawab

Pertama, suhu dibaca menggunakan sensor DHT:

```
float suhu = dht.readTemperature();
```

Kemudian kondisi pengiriman Request diubah menjadi:

```
if ((WiFi.status() == WL_CONNECTED) && (suhu > 35)) {
  WiFiClient client;
  HTTPClient http;
  http.begin(client, serverName);
  int httpResponseCode = http.GET();
  Serial.print("HTTP Response code: ");
  Serial.println(httpResponseCode);
  http.end();
}
```

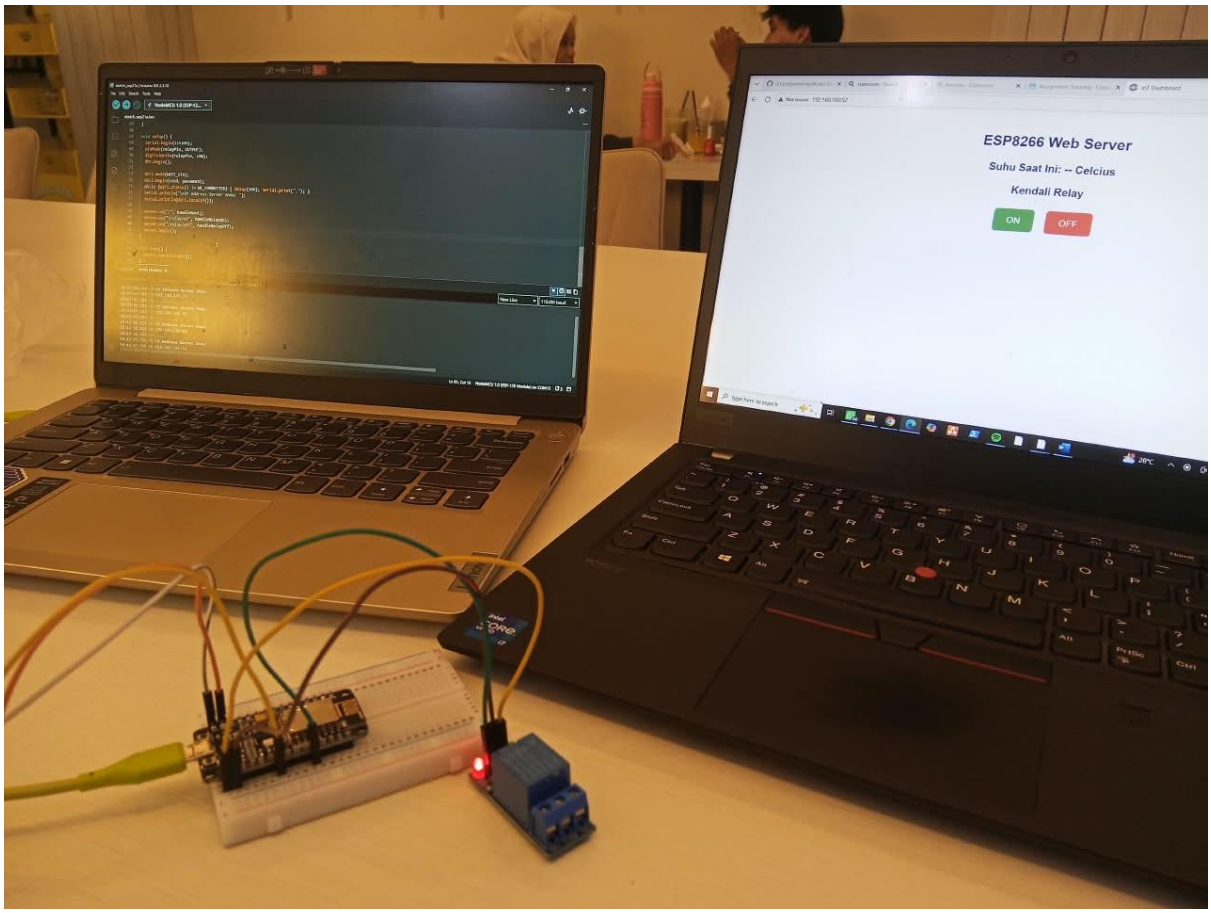
Penjelasan

Alur kerjanya adalah NodeMCU membaca suhu terlebih dahulu. Setelah itu program mengecek dua kondisi, yaitu Wi-Fi harus terhubung dan suhu harus lebih dari 35°C. Jika kedua kondisi terpenuhi, Client mengirim GET Request ke Server. Jika suhu masih 35°C atau kurang, Client tidak mengirimkan Request.

Jadi, kondisi utama yang digunakan adalah:

```
if ((WiFi.status() == WL_CONNECTED) && (suhu > 35))
```

Tugas: Pengembangan *Web Server* Cerdas dan Analisis Arsitektur



1. Langkah pengerjaan

1. Buka kode NodeMCU Server Latihan 2

Program Web Server dari latihan sebelumnya dibuka kembali.

Cari bagian kode yang berisi variabel index_html.

Di dalam bagian <head> HTML, tambahkan kode:

<meta http-equiv="refresh" content="5">

Contoh penempatannya:

```
<head>
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <meta http-equiv="refresh" content="5">
  <title>IoT Dashboard</title>
  <style>
    ...
  </style>
</head>
```

Setelah perubahan selesai, program disimpan.

Program kemudian di-upload kembali ke NodeMCU Server.

Setelah upload berhasil, alamat IP Server dibuka melalui browser.

Halaman Web Dashboard dibiarkan terbuka selama kurang lebih 20 detik.

Perubahan halaman diamati untuk mengetahui apakah proses refresh berjalan.

Tombol pengendali LED juga dicoba kembali untuk memastikan fitur ON dan OFF masih dapat digunakan.